Debreceni SZC Mechwart András Gépipari és Informatikai Technikum

Szoftverfejlesztő és -tesztelő

Vizsgaremek

Készítette: Karika Zoltán
Andrési György

2026.

Tartalomjegyzék

Tartalomjegyzék 2
1. Bevezetés 3
1.1. Témaválasztás indoklása 3
1.2. Kitűzött célok 3
2. Csapatmunka megvalósítása 4
3. Felhasználói dokumentáció 5
3.1. Az alkalmazás telepítése 5
3.2. Az alkalmazás indítása 6
3.3. Az alkalmazás használata 6
4. Fejlesztői dokumentáció 16
4.1. Fejlesztői környezet és technológiák 16
4.2. Adatbázis modell 20
4.3. Kiemelt kódrészletek 21
4.4. Tesztelés és hibajavítás 26
5. Összegzés 28
6. Továbbfejlesztési lehetőségek 29
7. Irodalomjegyzék 31

FixIT

1. Bevezetés

1.1. Témaválasztás indoklása

A **FixIT** egy vállalati jegykezelő rendszer, amely informatikai problémák bejelentésére és kezelésére szolgál. A projektet két fős csapatban valósítottuk meg. Az alkalmazás ötlete Zoltántól származik, aki munkája során hasonló környezetben szerzett tapasztalatokat.

A rendszer célja egy olyan webes felület kialakítása volt, amely lehetővé teszi a felhasználók számára problémáik egyszerű bejelentését ticketek létrehozásával, valamint az adminisztrátorok számára ezek kezelését és nyomon követését. Az alkalmazás tartalmaz továbbá egy chatbot funkciót is, amely segíthet a gyakori problémák gyors megoldásában és az adminisztrátori erőforrások tehermentesítésében.

.

1.2. Kitűzött célok

A cél egy egyszerűen kezelhető, intuitív felhasználói felület kialakítása volt, reszponzív működéssel amely lehetővé teszi a hibabejelentések gyors rögzítését és kezelését.

Emellett célunk volt egy stabil és biztonságos backend rendszer kialakítása, amely biztosítja a jegyek kezelését, a felhasználók jogosultságainak elkülönítését, valamint a rendszer megbízható működését. Szerettünk volna egy rendszert amelyben a felhasználók egyszerűen jelenthetik be problémáikat, illetve megfelelő adminisztrátori funkciókkal is rendelkezik és megoldott a két szerepkör közötti kommunikáció is.

A rendszer kialakításakor törekedtünk a későbbi bővíthetőség és továbbfejleszthetőség biztosítására.

2. Csapatmunka megvalósítása

A csapaton belüli együttműködést több kommunikációs eszköz segítségével biztosítottuk. A projekt során végrehajtott változtatásokat a GitHub verziókezelő és kollaborációs platform használatával követtük nyomon. A kisebb ötleteket és gyors egyeztetéseket Messenger alkalmazáson keresztül osztottuk meg egymással, míg a részletesebb megbeszélésekhez kéthetente rendszeres online egyeztetéseket tartottunk Microsoft Teams használatával.

A feladatmegosztás során Zoltán elsősorban a backend fejlesztéséért felelt, míg én főként a frontend megvalósításában vettem részt. A fejlesztés során ugyanakkor folyamatosan figyelembe vettük egymás javaslatait, és kölcsönösen támogattuk egymás munkáját.

Az adatbázis alapstruktúráját is Zoltán készítette el.

3. Felhasználói dokumentáció

3.1. Az alkalmazás telepítése

Python, Node.js és Git telepítése szükséges!

Python: https://www.python.org/downloads/

Node.js: https://nodejs.org/en/download

Git: https://git-scm.com/install

-1.github repository klónozása

git clone https://github.com/ZoltanKarika/FixITDesk

-2.Virtuális környezet létrehozása

py -m venv .(vagy py -m venv célmappa)

-3.Virtuális környezet aktiválása

scripts\activate(Windows esetén)

-4.Backend függőségek telepítése(manage.py „szintjén” futtatni)

pip install -r requirements.txt

-5.API Key kérése, környezeti változó beállítása .env file-ba

Mistral AI regisztráció(https://mistral.ai/), API_KEY kérése. Környezeti változó beállítása egy .env file-ban a .env.sample file alapján, ugyanazon a „szinten” a fileszerkezetben.

Megjegyzés: A projekt könnyebb ellenőrizhetősége és az azonnali futtathatóság érdekében a .env fájl (amely a külső API kulcsot tartalmazza) kivételesen a feltöltött állomány részét képezi. A fejlesztők tisztában vannak azzal, hogy éles üzemű környezetben a kulcsokat tartalmazó fájlok feltöltése a verziókezelőbe súlyos biztonsági kockázatot jelent, és a .gitignore fájl használata alapkövetelmény. Jelen

esetben a fájl mellékelése kizárólag a vizsgabizottság munkájának támogatását, a rendszer konfigurálás nélküli kipróbálhatóságát szolgálja. Valós környezetben a beállításokat a mellékelt .env.sample minta alapján, egyedileg kellene elvégezni. **Tehát 5. pont elhagyható ebben az esetben!**

-6.Adatbázis létrehozása

py manage.py migrate (manage.py szintjén)

A projekt könnyebb ellenőrizhetősége és az azonnali futtathatóság érdekében feltöltöttünk adatokkal egy adatbázist, ezért ezt nem kell végrehajtani.Valós környezetben az 5. ponthoz hasonlóan gitignore file-ba kerülne az adatbázisfile.

Tehát 6. pont elhagyható ebben az esetben!

-7.Frontend függőségek telepítése

fixit-frontend mappába(package.json „szintjén” futtatni)

cd fixit-frontend

npm install

3.2. Az alkalmazás indítása

1.Backend szerver indítása:

Virtuális környezet aktiválása:

scripts\activate

ezután a backend szerver indítása(localhost:8000)

py manage.py runserver

2.Frontend szerver indítása:

npm start(localhost:3000)

3.3. Az alkalmazás használata

1. Register - Login / Regisztráció - Bejelentkezés:

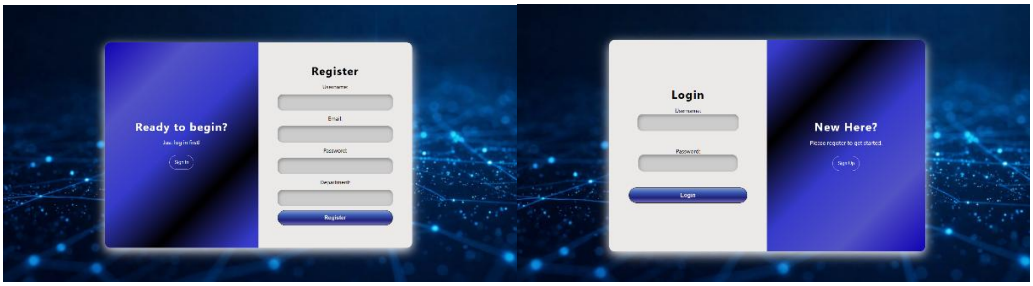
Az alkalmazás a regisztráció és az azt követő bejelentkezés után válik használhatóvá. Az űrlapokat értelemszerűen kell kitölteni:

Regisztráció:

- felhasználónév
- email
- jelszó(min 8 karakter)
- részleg

Bejelentkezés:

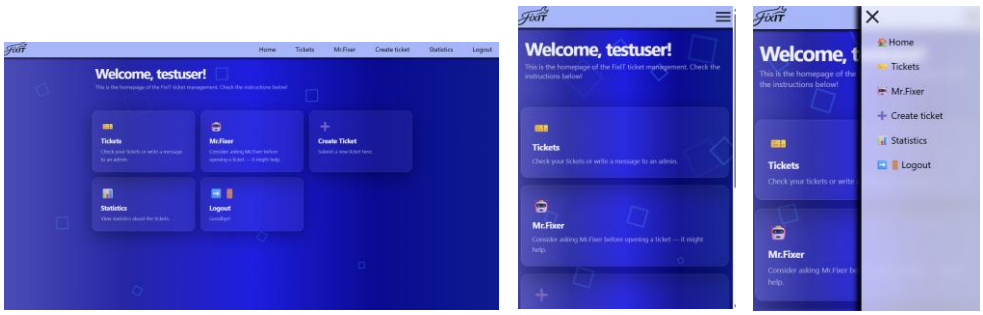
- felhasználó
- jelszó



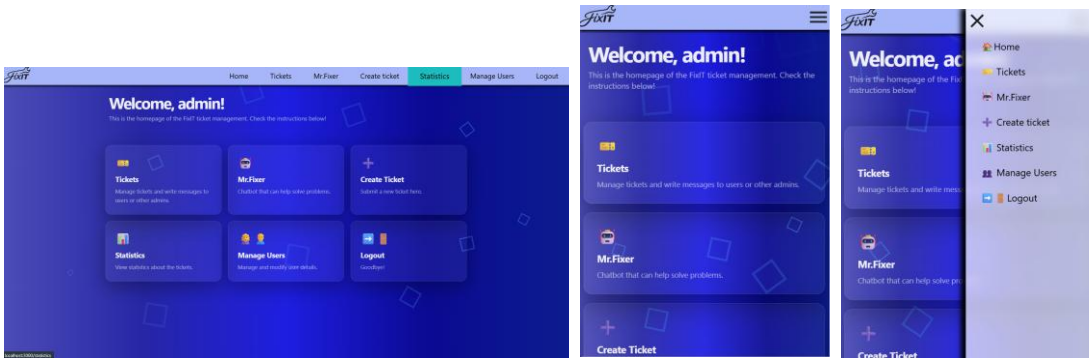
a regisztrációs és bejelentkezési oldal

2.Home / Kezdőoldal:

A kezdőoldalról navigálhat el a felhasználó a különböző funkciók saját oldalaira. Ezen kívül egy a program egészében elérhető navigációs sáv is segíti a program használatát, az ablak átméretezése után elérhető reszponzív mobil nézetben ez átvált egy opcionális oldalsó menüre. Az elérhető funkciók jogosultsági szinthez kötöttek egy nem-admin szintű felhasználó lehetőségei korlátozottak.



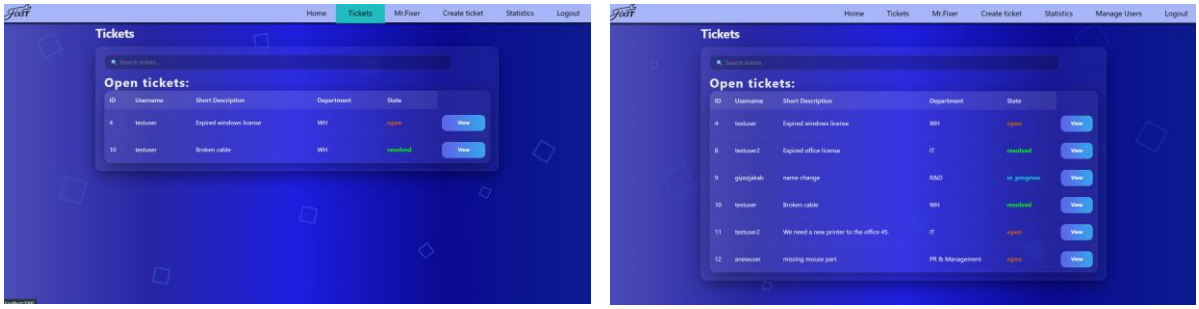
kezdőoldal és kezdőoldal mobil nézetben



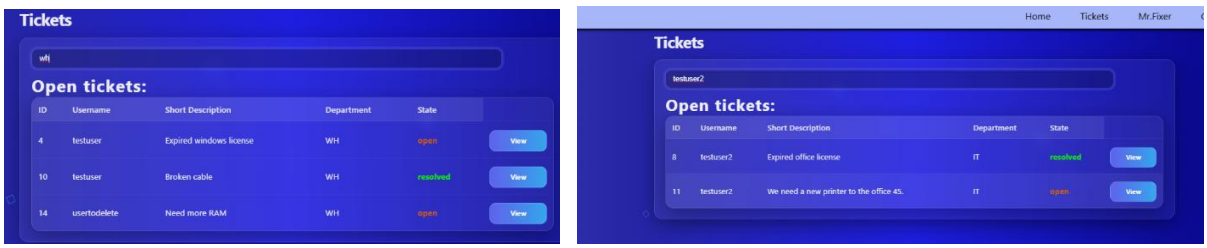
admin felhasználó esetén a Manage Users fül is elérhető, továbbá az oldalakon belül is több funkció válik elérhetővé

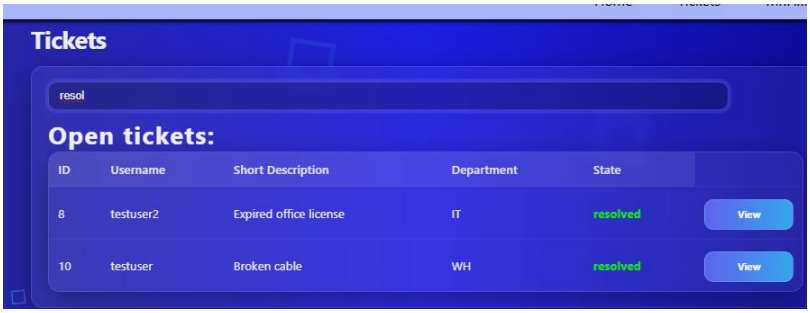
3.Tickets / Jegyek:

A Tickets fülön érhető el a létrehozott jegyek listája. Nem-admin szintű felhasználó csak a saját jegyeit látja. Admin jogú felhasználó minden felhasználó jegyeit elérheti. Az itt található view button segítségével léphetünk be a Ticket view felületbe, ahol a megjegyzés funkció is elérhető. Egy keresősáv könnyíti, gyorsítja a jegyek kezelését, keresését és beazonosítását. Segítségével lehet szűrni a jegyek tulajdonságaira úgy mint státusz, részleg, leírás vagy felhasználó.



nem-admin felhasználó csak a saját jegyeit látja, míg egy admin mindenkiét

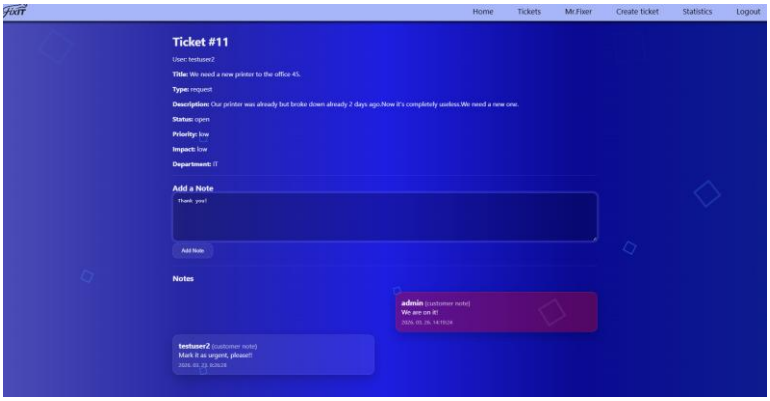




szűrés különböző feltételek alapján

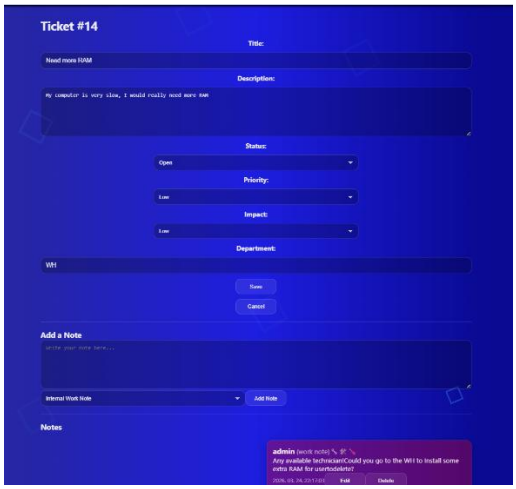
4.Tickets view / Jegyek belső nézet:

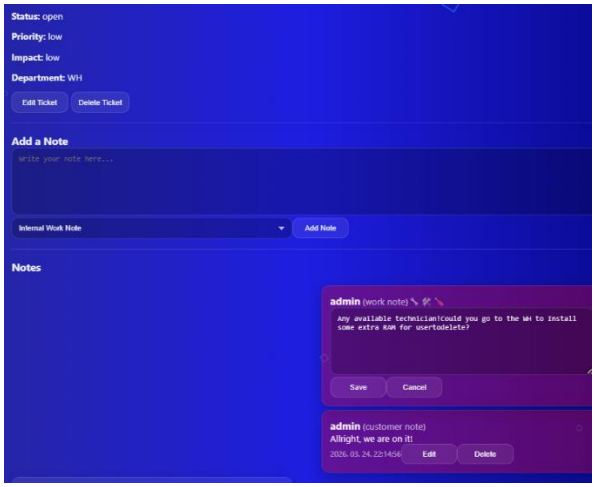
A Tickets fülön kilistázott jegyek „view” gombjára klikkelve válik elérhetővé a jegyek belső nézete. Itt látható az összes adat az adott jegyről, és itt válik elérhetővé a megjegyzések felület, amivel a felhasználó megjegyzéseket adhat az adminnak, illetve fogadhat.



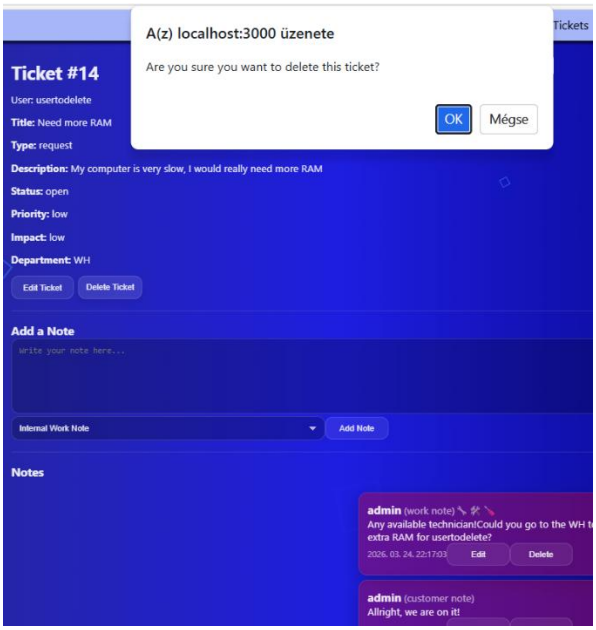
nem-admin felhasználó üzeneteket küldhet és fogadhat az adminoktól

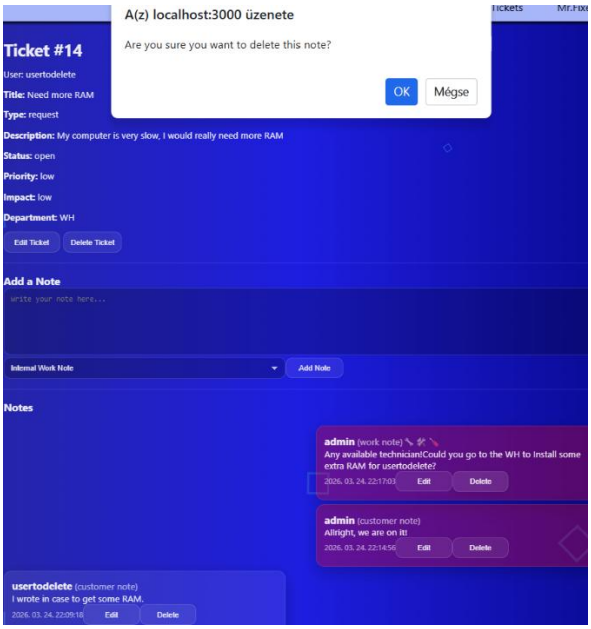
Az admin jogosultsággal rendelkező felhasználó nem csak látja, de módosíthatja a jegyek valamennyi adatát jegy módosítása gombbal, törölheti a jegyet vagy akár módosíthatja a megjegyzéseket is. Lehetősége van továbbá választani a felhasználói megjegyzések és belső megjegyzés között, amivel a csak adminok által látható munkával kapcsolatos megjegyzéseket hagyhat.



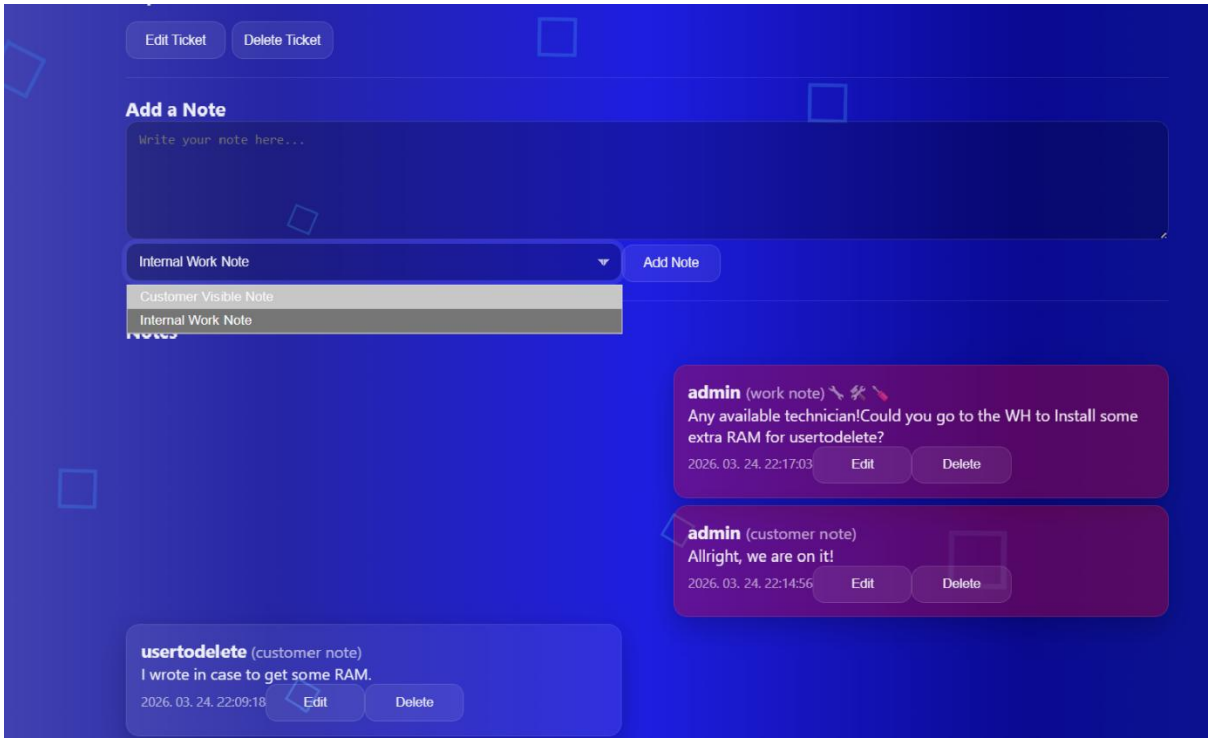


admin jogosultsággal lehet módosítani a jegyeket és a hozzájuk tartozó megjegyzéseket is



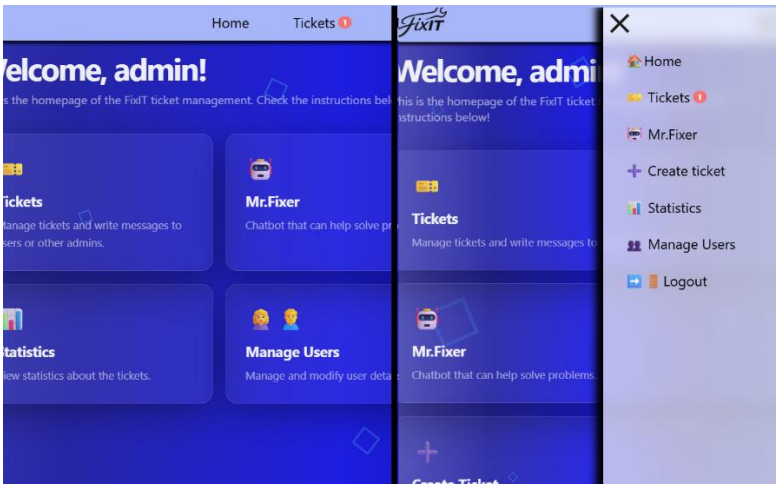


admin törölheti is a jegyeket és a hozzájuk tartozó megjegyzéseket

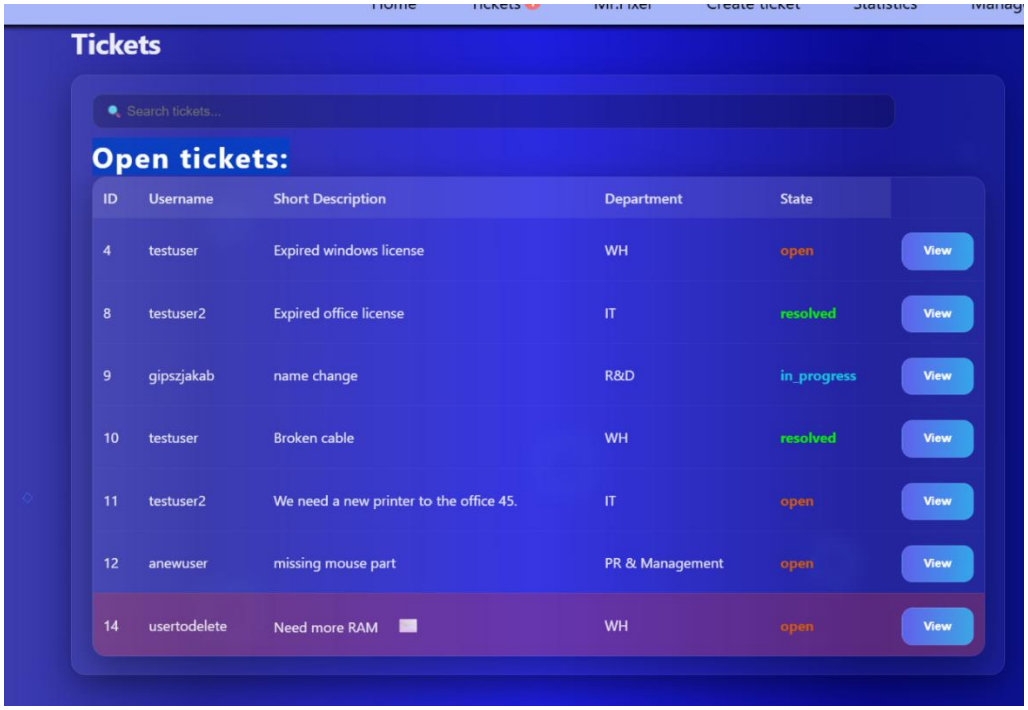


admin hagyhat belső megjegyzéseket, amit kizárólag csak más adminok láthatnak, ezeket az üzeneteket egy kis szerszám-ikon is jelzi

Minden felhasználó értesítést kap, ha megjegyzést adtak valamelyik jegyéhez, **adminok mindenről kapnak értesítést**!



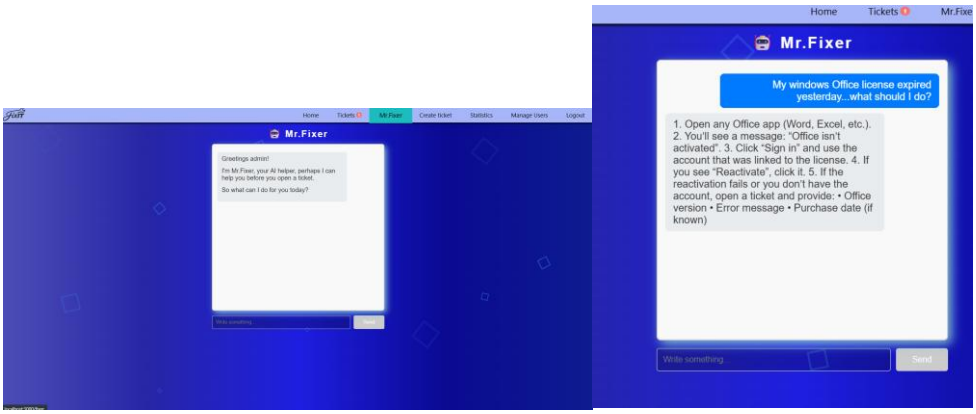
az értesítések a navigációs sávban illetve a mobil nézet oldalsó menüjén is megjelennek



az értesítések a Tickets fül listájában is megjelennek

5. Mr.Fixer / Chatbot:

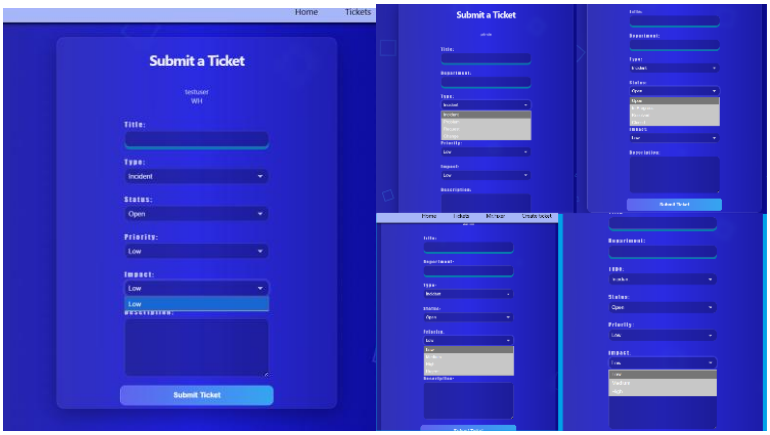
Minden felhasználó számára elérhető egy AI chatbot, amelynek feladata hogy az egyszerűbb problémákra megoldást találjon illetve segítsen megoldani mielőtt a felhasználók jegyet nyitnának. Természetesen egy chatbot nem tud minden problémát megoldani de az elsődleges prioritása az adminisztrátori erőforrások tehermentesítése, adminok segítése. A **navigációs sávon is ezért lett tudatosan a jegy létrehozása opció elé helyezve.**



a chatbot segíthet az adminok tehermentesítésében

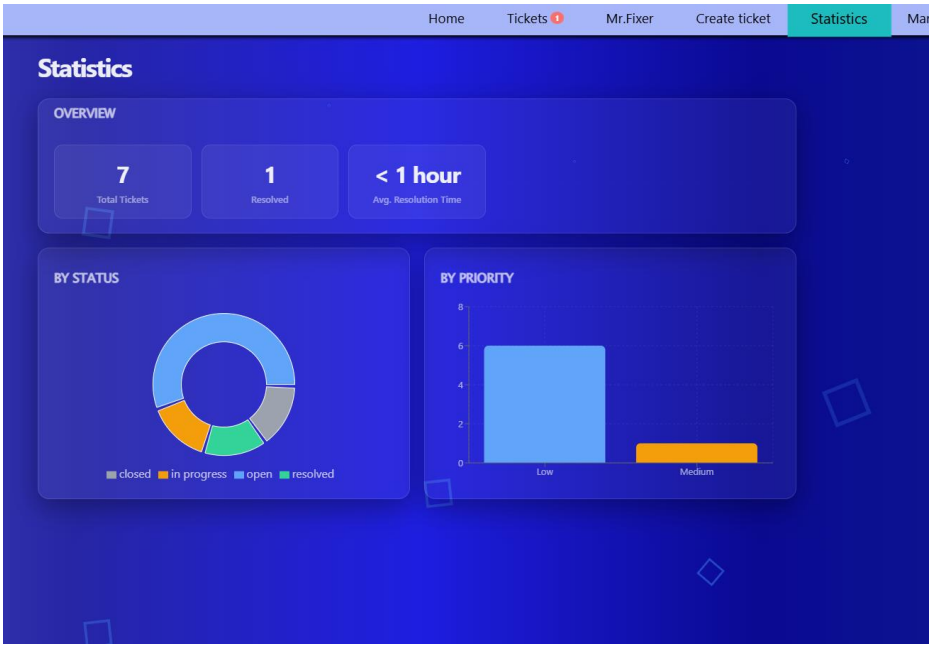
6.Create ticket / Jegy létrehozása:

Minden felhasználó nyithat jegyet, de a nem-admin felhasználók csak a legalacsonyabb értékekkel, prioritással. Amennyiben sürgősebb az eset, írhatnak az adminoknak, akik elbírálhatják. Admin jogosultsággal bármilyen ticket nyitható.



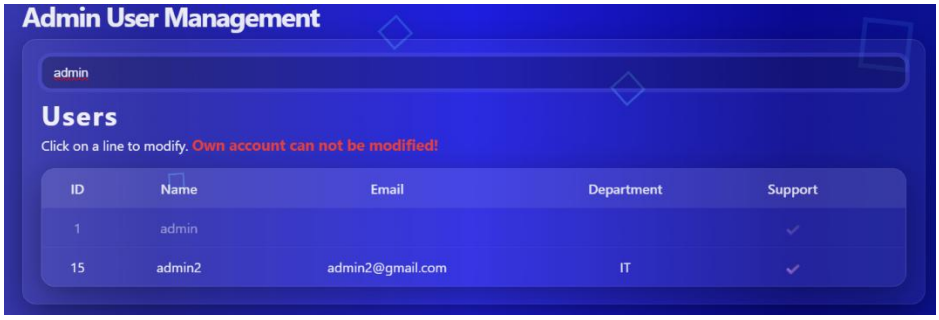
7.Statistics / Statisztikák:

A felhasználóknak lehetőségük van egyszerűbb statisztikákat nézni a jegyekről azok adatai alapján. Például prioritás, státusz vagy átlagos megoldási idő alapján. Normál felhasználó csak a saját jegyeinek adatait látja, míg az adminok mindenkiét.

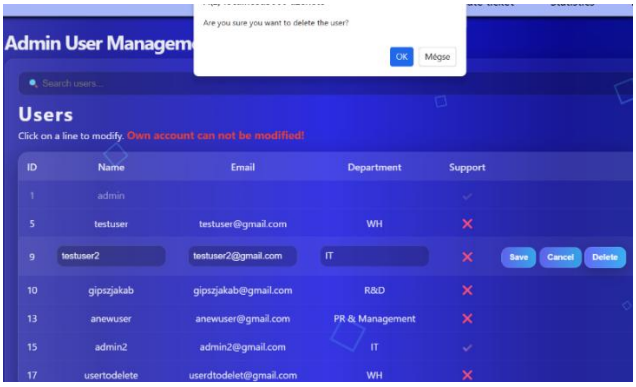


8.Manage Users / Felhasználók kezelése:

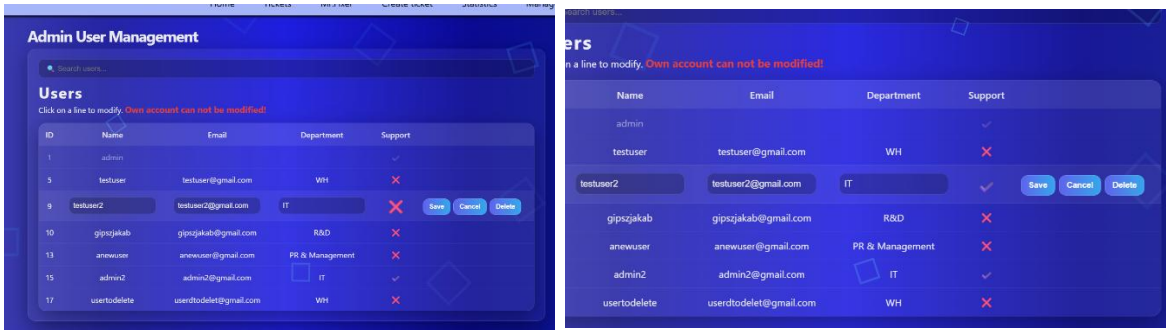
Ez az opció teljesen admin exkluzív, ezen a fülön a felhasználók adatai módosíthatóak, felhasználók törölhetőek vagy akár admin jogosultság is adható. Saját adatok módosítása le van tiltva ezen a belső felületen, ütközések elkerülése miatt. A felhasználók kezelése fül is el van látva egy keresővel, amellyel a felhasználók különböző adataira kereshetünk rá vagy szűrhetünk.



szűrhetünk különböző adatokra



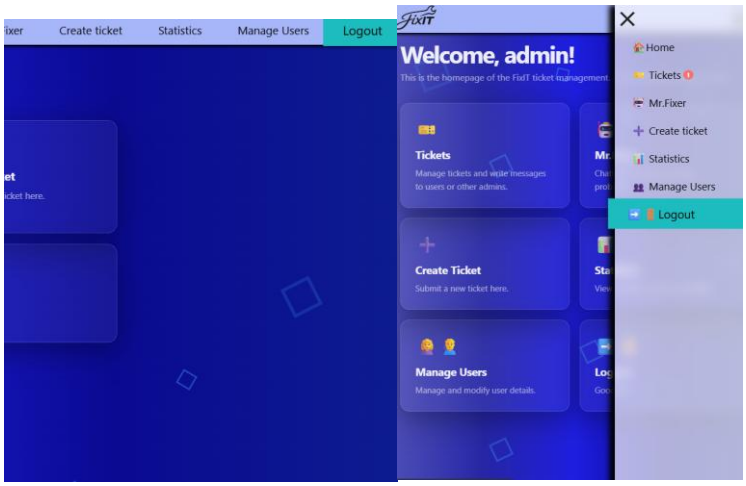
admin joggal törölhetünk felhasználót



admin felhasználói jogosultsággal módosíthatjuk a felhasználók adatait, admin jogot adhatunk

9.Logout / Kijelentkezés:

Az utolsó opcióval jelentkezhetünk ki, ez is, mint minden egyéb opció, a navigációs sávon és az oldalsó menün is elérhető.



4. Fejlesztői dokumentáció

4.1. Fejlesztői környezet és technológiák

A program fejlesztéséhez az alábbi eszközöket használtuk:

Frontend:

-Node.js: A Node.js szolgált fejlesztői környezetként a függőségek kezeléséhez és adott alapot a hatékony fejlesztéshez.

-Javascript(ES6+): Választásunk azért esett erre a nyelvre, mert az eseményvezérelt, aszinkron működési modellje (async/await) lehetővé teszi a reszponzív felhasználói élményt: az adatok lekérése a háttérben zajlik, így az interfész soha nem fagy le várakozás közben. A modern nyelvi elemek (pl. ternary, spread operator) használata pedig moduláris és könnyen tesztelhető kódbázist eredményezett.

-HTML5 és CSS3: Az alkalmazás szemantikus felépítéséhez a legmodernebb HTML5 szabványt alkalmaztuk.Ez nem csak biztosítja a dokumentum logikai struktúráját, de elengedhetetlen a keresőoptimalizálás és a képernyőolvasó szoftverek támogatásához. A vizuális megjelenítésért és a felhasználói felület (UI) konzisztenciájáért a CSS3 felel. A modern elrendezési modellek segítségével alakítottuk ki a komplex dashboard-elemeket, illetve adtunk stílust a megjelenésnek.

- React: A React library komponensalapú architektúrája lehetővé tette a kód újrahasznosíthatóságát és a felület moduláris felépítését.Segítségével gyors és reszponzív felhasználói élmény érhető el, ami kritikus az adatok dinamikus megjelenítésekor.

-Js-cookie: Ezt a JavaScript könyvtárat a sütik (cookies) kezeléséhez használtuk. Biztonságos és egyszerű felületet biztosít a hitelesítési tokenek kiolvasásához és törléséhez kijelentkezéskor.

-**React-charts:** A statisztikai adatok megjelenítésérevélasztottuk a react charts könyvtárat, mert natívan integrálódik a React életciklusába, és képes a backendről érkező komplex adathalmazokat látványos, interaktív grafikonokká alakítani.

Backend:

-Python: A nyelv tiszta szintaxisa és hatalmas ökoszisztémája felgyorsította a fejlesztést. Kézenfekvő volt terjedelmes dokumentációja és támogatottsága miatt is.

-Django: A Django keretrendszerre esett a választásunk a „batteries included" tulajdonsága miatt, segítségével egyszerűen lehet professzionális admin felületet, hitelesítési rendszert vagy akár URL-kezelést megoldani, létrehozni. Beépített ORM-rendszere lehetővé tette hogy az adattáblákat python osztályokként kezeljük.

-Django REST framework: A DRF használatával szabványos, jól dokumentált és könnyen tesztelhető RESTful API végpontokat alakíthattunk ki.

-**Django CORS headers:** Elengedhetetlen a frontend és a backend elkülönült futtatásakor (cross-origin) hogy szabályozott és biztonságos módon engedélyezzük az erőforrások elérését a böngésző számára.

-Simple JWT: Ez a könyvtár biztosította az állapotmentes hitelesítést, ami skálázhatóvá teszi a rendszert. A tokenek használata egyszerűbbé teszi a kliens és a szerver közötti biztonságos kommunikációt.

-Requests library: Egyszerű és hatékony HTTP-kliens, amelyet külső szolgáltatással való kommunikációra használtunk, kiváltva ezzel egy bonyolultabb megoldást. A chatbot feature ezzel meglepően egyszerűen megoldhatóvá vált.

-Decouple library: A biztonságos fejlesztés alapkövetelménye, hogy az érzékeny adatokat (API_KEY, adatbázis jelszavak) elkülönítsük a forráskódtól környezeti változók segítségével. A python-decouple használata nélkül a projekt érzékeny adatai (pl.külső LLM API key-e) közvetlenül a forráskódba kerültek volna.

Biztonsági megjegyzés a vizsgamunkához:

Az alkalmazás konfigurációjához szükséges .env fájl a vizsgamunka részeként feltöltésre került, amely tartalmazza a külső LLM szolgáltatáshoz (API) tartozó kulcsot.

Fontos hangsúlyozni, hogy ez kizárólag a vizsgabizottság munkájának megkönnyítését és az alkalmazás azonnali kipróbálhatóságát szolgálja. Éles környezetben és valós verziókezelési folyamat során a .env fájl a .gitignore szabályai szerint titkos marad. Ilyen esetben a forráskód mellé egy .env.sample fájlt mellékelünk(ezt mi is megtettük), amely alapján az üzemeltető a saját hitelesítő adataival konfigurálhatja a rendszert.

Adatbázis:

-SQLite: Az adatbázis-motor kiválasztásakor az SQLite mellett döntöttünk, mivel a Django ORM-en keresztül az adatbázis-kezelés motorfüggetlen. Fájlalapú felépítése miatt nem igényel külön szerverüzemeltetést, ami ideálissá tette a gyors fejlesztéshez és a hordozhatósághoz.

Verziókezelés:

-Git és GitHub:

A Git lehetővé tette a forráskód verzióinak követését, a különböző fejlesztési fázisok elkülönítését és a biztonságos visszaállási pontok létrehozását.A GitHub egy kitűnő felület és kollaborációs tér ennek kiegészítésére, amely jelentősen megkönnyítette a csapatmunkát.

HTML
CSS
JS

python
+
django

SQLite

git

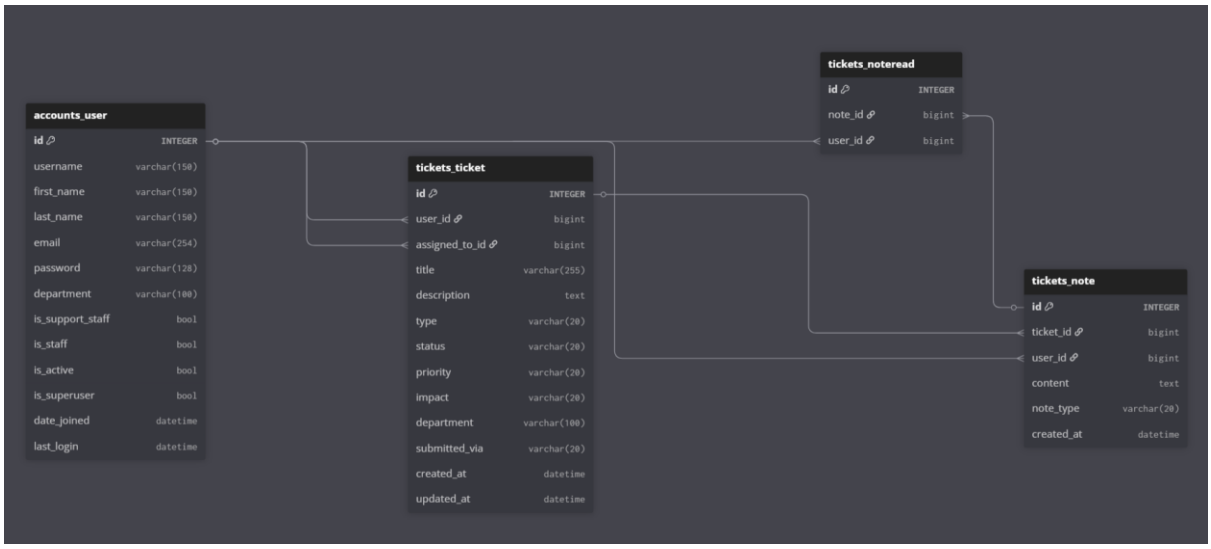
GitHub

4.2. Adatbázis modell

Az adatbázis modellünk alapja a Django beépített user modellje,amit kiterjesztettünk pár extra mezővel mint a részleg(department) és a saját admin jog(is_support_staff) az AbstractUser osztályának segítségével. Override metódussal minden kreált superuser automatikusan admin jogot kap.

```
def save(self, *args, **kwargs):
    if self.is_superuser:
        self.is_support_staff = True
    super().save(*args, **kwargs)
```

Az ebből származó accounts_user táblához csatlakozik idegen kulccsal a tickets_ticket tábla, amihez hasonló módon viszonyul a tickets_note tábla, ez a tábla idegen kulccsal csatlakozik a userhez is. Egy user törlésénél a hozzá tartozó ticket és ahhoz tartozó note is törlődik. A tickets_noteread kapcsolótáblának az értesítések megjelenítése miatt van szerepe, idegen kulccsal csatlakozik note és user-hez is.



4.3. Kiemelt kódrészletek

Néhány kódrészlet kiemelt szerepet kapott a fejlesztés során.

Ilyen például az **api.js** DRY-elv(**Don't Repeat Yourself**) szellemében íródott kódsor, ami az api kéréseink „lelke", paraméterezhetősége miatt szinte minden kérésnél újrahasznosítható, ez helyezi el a kérésekbe a CSRF-token-t, illetve ez kezeli a refresh-folyamatot is, az access token lejárata után.

```
function getCSRFToken() {
  return Cookies.get('csrftoken') || '';
}
                                  (parameter) method: string
async function request(path, { method = 'GET', body = null, headers = {}, ...customOptions } = {}) {
  const options = {
    method,
    credentials: 'include',
    headers: {
      'Content-Type': 'application/json',
      ...headers,
    },
    ...customOptions,
  };

  if (['POST', 'PUT', 'PATCH', 'DELETE'].includes(method.toUpperCase())) {
    options.headers['X-CSRFToken'] = getCSRFToken();
  }

  if (body) {
    options.body = JSON.stringify(body);
  }

  let response = await fetch(`${API_URL}${path}`, options);

  if (response.status === 401) {
    const refreshResponse = await fetch(`${API_URL}/api/token/refresh/`, {
      method: 'POST',
      credentials: 'include',
    });
    console.log("REFRESHED");

    if (refreshResponse.ok) {
      response = await fetch(`${API_URL}${path}`, options);
    } else {
      throw new Error('Session expired');
    }
  }

  return response;
}

const api = {
  get: (path, options = {}) => request(path, { ...options, method: 'GET' }),
  post: (path, body, options = {}) => request(path, { ...options, method: 'POST', body }),
  patch: (path, body, options = {}) => request(path, { ...options, method: 'PATCH', body }),
  put: (path, body, options = {}) => request(path, { ...options, method: 'PUT', body }),
  delete: (path, options = {}) => request(path, { ...options, method: 'DELETE' }),
};

export default api;
```

Ezt használva egy kérés egy egyszerű import-al és paraméteradással meghívható:

```
console.log("Sending form data:", regFormData);
let response = await api.post('/api/accounts/register/', regFormData);
if (!response.ok) {
  throw new Error("Registration failed");
}
```

Használtuk ezt **checkUser** metódusunknál is, amit többször is újrahasznosítottunk a hitelesítési folyamatok közben:

```
const checkUser = async () => {
  try {
    let response = await api.get('/api/accounts/whoami/');
    if (!response.ok) {
      navigate('/gatekeeper');
      logoutHandler();
      return;
    }
    const data = await response.json();
    setUserInfo(data);
```

Ez a hívás **WhoAmIView** segítségével ad vissza user adatokat:

```
class WhoAmIView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request):
        return Response({''
        'id': request.user.id,
        'username': request.user.username,
        'is_support_staff' :request.user.is_support_staff,
        'department': request.user.department})
```

A fenti példában szereplő kérést, és az abból származó adatokat csak megjelenítésre használtuk, tehát hiába kapunk vissza a kérésből pl. is_support_staff = True értéket, a tényleges kérések backend-en biztonságosan védve vannak.

A React useContext hookja és egy egyedi **UserProvider** komponens segítségével hatékonyan tudtuk kezelni a globális felhasználói állapotot. Ez a megoldás lehetővé tette, hogy az aktuális (nem kritikus) felhasználói adatokat a teljes komponensfán elérhetővé tegyük. A Provideren belül implementáltuk a localStorage szinkronizációt is, így a perzisztens tárolás (mentés és törlés) logikája egy helyen összpontosul. Miután az alkalmazás gyökerét (index.js) körbezártuk a Providerrel, bármely alkomponensben egyszerűen hozzáférhetünk a felhasználói állapothoz és a kapcsolódó függvényekhez.

```
const UserContext = createContext();

export const UserProvider = ({ children }) => {
    const [unreadCount, setUnreadCount] = useState(0);
    const [user, setUser] = useState(() => {
        try {
            return JSON.parse(localStorage.getItem('user')) || null;
        } catch {
            return null;
        }
    });

    useEffect(() => {
        if (user) {
            refreshUnreadCount();
        }
    }, [user]);

    const refreshUnreadCount = async () => {
        if (!user) return;
        try {
            const res = await api.get('/api/tickets/');
            if (res.ok) {
                const data = await res.json();
                const total = data.reduce((sum, t) => sum + (t.unread_count || 0), 0);
                setUnreadCount(total);
            }
        } catch (err) {
            console.error(err);
        }
    };

    const loginHandler = (userData) => {
        setUser(userData);
        localStorage.setItem('user', JSON.stringify(userData));
        console.log(user);
    };

    const logoutHandler = () => {
        setUser(null);
        localStorage.removeItem('user');
    };

    return (
        <UserContext.Provider value={{ user, setUser, loginHandler, logoutHandler, unreadCount, refreshUnreadCount}}>
            {children}
        </UserContext.Provider>
    );

};

export const useUserHandler = () => useContext(UserContext);
```

Egy másik feature, ami említésre érdemes, **a Mr.Fixer, a chatbot** funkciónk.

Ennek használatához először is regisztrálnunk kell egy LLM szolgáltató oldalán, a mi esetünkben a MISTRAL AI-ra esett a választás, mivel az egyik legmegbízhatóbb LLM szolgáltatás.

regisztráció: https://mistral.ai/

AI Studio nevezetű szolgáltatásukkal ingyenes API-KEY kérhető, amivel egy egyszerű HTTPS kéréssel küldhetünk üzenetet és kaphatunk választ. Fontos hogy a megfelelő, a szolgáltatás által elvárt formátumban kell a kérést küldenünk.

```python
def aichat(request):
    try:
        data = json.loads(request.body)
        user_message = data.get('message', '')
        if not settings.MISTRAL_API_KEY:
            return JsonResponse({
            "error": "API OR .env FILE IS MISSING - MR.FIXER IS DOWN ☹️"
        }, status=503)


        API_KEY = settings.MISTRAL_API_KEY
        API_URL = "https://api.mistral.ai/v1/chat/completions"

        headers = {
            "Content-Type": "application/json",
            "Authorization": f"Bearer {API_KEY}"
        }

        system_prompt = (
            "You are Mr.Fixer, a chatBot for a ticket managing system."
            "Your purpose is to give help to the users before they open a ticket to save resources."
            "Your responses must be concise and professional, delivered in a practical 'mechanic' style. "
            "Always provide actionable, hands-on solutions to technical problems."
            "You have to presume that the users don't have special skills like soldering, electrical engineering etc."
            "So if something is broken for example, recommend to open a ticket."
            "File attaching is not handled, so do not ask them to attach files, pics etc."
            "Respond in the same language the user uses."
        )

        payload = {
            "model": "mistral-small-latest",
            "messages": [
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": user_message}
            ],
            "temperature": 0.7
        }

        response = requests.post(API_URL, json=payload, headers=headers, timeout=30)
        res_data = response.json()


        if "error" in res_data:
            return JsonResponse({"error": res_data["error"]["message"]}, status=400)

        ai_text = res_data['choices'][0]['message']['content']

        return JsonResponse({"reply": ai_text})

    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)
```

Itt adjuk meg az API-key-t, headerben

Mistral által elvárt formátum, a model típusa, az üzenet "előüzenettel", valamint a temperature, ami az AI kreativitását állítja

Mistral válaszának "kicsomagolása" és átalakítása

A **request könyvtár** használatával egyszerűen küldhetünk https kérést, mint fentebb is látható.

A **decouple** környezeti változó kezelő **könyvtár**ral megtehetjük hogy nem direktben, hanem egy .env file-ból olvassuk be az API-KEY-t, amit nem töltünk fel verziókezelővel semmilyen platformra, ezáltal minden kényes információ biztonságban van a backend-en.

Egy másik érdekes kódrészlet a **TicketListCreateView** amivel a jegyeinket listázzuk ki. Ennek segítségével alapszintű felhasználható eléri a saját jegyeit, admin jogosultságú pedig mindenkiét. Tartalmaz tovább egy részletet amely a „closed” státuszú jegyeket törli a listából, utolsó módosítás után 21 nappal.

```
class TicketListCreateView(generics.ListCreateAPIView):
    serializer_class = TicketSerializer
    permission_classes = [permissions.IsAuthenticated]

    def get_queryset(self):
        user = self.request.user
        queryset = Ticket.objects.all()

        if user.is_support_staff:
            limit = timezone.now() - timedelta(days=21)
            Ticket.objects.filter(
                status='closed',
                updated_at__lt=limit
            ).delete()

        if user.is_support_staff:
            status = self.request.query_params.get('status', None)
            if status:
                queryset = queryset.filter(status=status)
        else:
            queryset = Ticket.objects.filter(user=user).exclude(status='closed')

        return queryset
    def perform_create(self, serializer):
        serializer.save(user=self.request.user)
```

a 21 napnál régebbi "closed" jegyeket törli

visszaad minden jegyet adminnak

else: user csak a saját jegyeit látja, a "closed" jegyeket nem

4.4. Tesztelés és hibajavítás

Nagy kihívást jelentett a komplex css hierarchia kialakítása, a modernebb animációk illetve a pozicionáló tag-ek sokszor ütköztek. A böngésző DevTools opciója nagy segítséget nyújtott a problémák beazonosításához.

Főként manuális tesztelési módszereket alkalmaztunk, úgy mint például az api kérések nagy részéhez „console.log” konzolparanccsal írattuk ki az aktuális értékeket a kérések eredményeinek, a problémák beazonosításának érdekében. A backend-en is alkalmazható volt a konzolra írás módszere, csak ott print paranccsal értük el ugyanezt.

A korábban már említett „whoami” sokat segített az azonosítási és hitelesítési funkciók fejlesztése közben.

```
const whoami = await api.get('/api/accounts/whoami/');
loginHandler(whoami.data);
```

```
const loginHandler = (userData) => {
    setUser(userData);
    localStorage.setItem('user', JSON.stringify(userData));
    console.log(user);
};
```

```
class WhoAmIView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request):
        return Response({''
        'id': request.user.id,
        'username': request.user.username,
        'is_support_staff' :request.user.is_support_staff,
        'department': request.user.department})
```

Ez többek között a loginHandler fgv-nek is ad adatokat, ami konzolra írva azt, sokat segített a rendszer aktuális állapotának felmérésében.

Egy másik korábban említett funkció, az automatikus jegytörlés is tesztelést igényelt, itt az időadatot írtuk át manuálisan a kód eredményességének megállapítása érdekében.

```python
if user.is_support_staff:
    limit = timezone.now() - timedelta(days=15)
    Ticket.objects.filter(
        status='closed',
        updated_at__lt=limit
    ).delete()
```

```python
if user.is_support_staff:
    limit = timezone.now() - timedelta(seconds=15)
    Ticket.objects.filter(
        status='closed',
        updated_at__lt=limit
    ).delete()
```

Illetve igyekeztünk minél több kérésnél try-catch szerkezeteket használni, ami nagy segítség a problémák lokalizásánál és kiiktatásánál.

```javascript
const refreshUnreadCount = async () => {
    if (!user) return;
    try {
        const res = await api.get('/api/tickets/');
        if (res.ok) {
            const data = await res.json();
            const total = data.reduce((sum, t) => sum + (t.unread_count || 0), 0);
            setUnreadCount(total);
        }
    } catch (err) {
        console.error(err);
    }
};
```

Backenden a python try-except módszerét szintén alkalmaztuk ugyanezen okból, a szerver értékes információkkal tudott szolgálni a fejlesztés hatékonyságának segítésében.

```python
try:
    validated_token = self.get_validated_token(access_token)
    return self.get_user(validated_token), validated_token
except Exception as e:
    raise exceptions.AuthenticationFailed(f"Invalid token: {str(e)}")
```

5. Összegzés

A fejlesztés során egy olyan modern, teljes körű (Full-Stack) webalkalmazást hoztunk létre, amely választ ad a kis- és középvállalkozások informatikai incidenskezelési és statisztikai igényeire. A projekt bebizonyította, hogy a Django robusztus backendje és a React dinamikus frontendje közötti szimbiózis lehetővé teszi skálázható és biztonságos rendszerek építését.

Tapasztalatok és tanulságok:

- **Biztonsági fókusz:** A fejlesztés egyik legnagyobb kihívása és egyben legértékesebb tanulsága a **HttpOnly cookie-alapú JWT hitelesítés** implementálása volt, amely magasabb szintű védelmet nyújt a kliensoldali támadásokkal szemben.
- **Adatvizualizáció:** A **Recharts** könyvtár integrálásával megtanultuk a nyers adatbázis-adatokat közérthető, üzleti döntéseket támogató grafikonokká formálni.
- **AI-integráció:** A **Mistral AI** beépítése megmutatta, hogyan emelhető egy hagyományos jegykezelő rendszer funkcionalitása modern, intelligens asszisztencia szolgáltatással.

A projekt sikeresen teljesítette a kitűzött célokat: egy átlátható, könnyen kezelhető és technikailag stabil rendszert alkottunk meg. A fejlesztés folyamata során mélyebb betekintést nyertünk az aszinkron programozásba, a relációs adatbázis-tervezésbe és a modern szoftverfejlesztési életciklusokba. A következő, 6.-os pont alapján a szoftver készen áll a jövőbeni bővítésekre és a piaci igényekhez való további alkalmazkodásra.

6. Továbbfejlesztési lehetőségek

-**Mobil kompatibilitás:**

Bár a fejlesztés során a reszponzív szemléletet (Flexbox, Media Queries) követtük, a jövőben cél az alkalmazás teljeskörű mobil integrációja.

-AI/chatbot fejlesztése:

A jelenlegi LLM-integrációt (Mistral AI) kiterjesztenénk egy kontextus-tudatos chatbotra. Az AI-val folytatott beszélgetések naplózása és elemzése segíthet a gyakori hibajelenségek automatikus felismerésében, az erőforrások optimalizálásában.

-Biztonsági fejlesztések:

A biztonság fokozása érdekében tervezzük a kód további optimalizálását, egyes részek biztonság szempontjából való újragondolását. Például localstorage-ben tárolunk bizonyos nem-kritikus adatokat(id, username), ezekre a jövőben lehetne még biztonságosabb alternatívákat keresni.

-Kommunikációs felület továbbfejlesztése:

A jegykezelő rendszer chat funkciójának továbbfejlesztése például egy valós idejű chat-modullal, amely lehetővé tenné az ügyfelek és a technikusok közötti azonnali üzenetváltást vagy akár a fájlmegosztást a platformon belül.

-UI továbbfejlesztése:

Light/Dark Mode bevezetése. A felhasználói visszajelzések alapján a felület további ergonómiai javítása, még felhasználóbarátabbá tétele.

-Deployment megvalósítása:

Az alkalmazás pubikálása, elérhetővé tétele nem lokális szerverről is.

-**Többnyelvű felület:**

Magyar-Angol vagy akár még több nyelvű opció elérhetősége, hogy minden felhasználó számára kezelhető legyen az alkalmazás, minél több felhasználó számára elérhető legyen.

7. Irodalomjegyzék

Péter Miklós YT csatornája: Miklós Péter - YouTube

SuperSimpleDev YT csatorna: https://www.youtube.com/@SuperSimpleDev

Python: https://www.python.org

Django: Django documentation | Django documentation | Django

React: https://react.dev/

GitHub: https://github.com

W3Schools oldalak: W3Schools Online Web Tutorials

Mistral AI: https://mistral.ai/

Claude: https://claude.ai

ChatGPT: https://chatgpt.com/